SEPA: plan de trabajo con ingesta en la PC

Oct 18, 2026 · @Nicolas

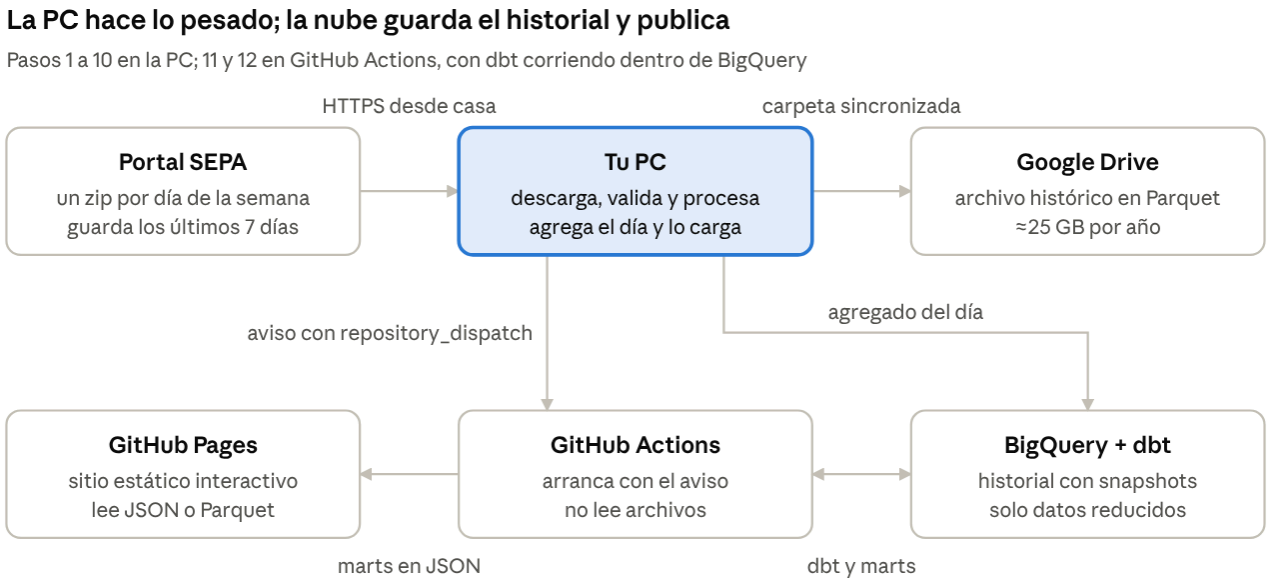
Resumen

La descarga diaria pasa a hacerse en tu PC. El resto del pipeline sigue en la nube, como en el plan original. El cambio se debe a que el portal SEPA responde 403 a GitHub Actions y a Google Colab, y 200 desde la PC de casa.

- **Tu PC (Programador de tareas de Windows):** descarga las fechas que falten, valida, genera los Parquet y los guarda en Drive. Después calcula el agregado del día, lo carga a BigQuery y le avisa a GitHub.
- **GitHub Actions:** cuando la PC avisa que hay datos nuevos, corre dbt dentro de BigQuery y publica el sitio. No lee archivos.
- **El objetivo se ajusta:** de "sin depender de una PC prendida" a "la PC tiene que prenderse al menos una vez cada 7 días", porque el portal guarda los últimos 7 días y el script se pone al día solo.

Las etapas 0 y 1 están hechas, y la ingesta en la PC (etapa 3) ya corre programada: falta una semana sin intervención para cerrarla. En la etapa 2 quedan la canasta, los datos de producto y cómo consulta datos el sitio.

Arquitectura



Arquitectura con ingesta en la PC · 6 piezas

La PC hace todo lo que necesita la conexión de casa o el detalle completo: descarga, procesa, archiva en Drive y carga a BigQuery solo el agregado del día. GitHub Actions no lee archivos: dbt corre dentro de BigQuery. Cloud Storage se descartó, porque solo hacía de puente.

Qué corre dónde

La PC hace los pasos 1 a 10, que necesitan la conexión de casa o el detalle completo. GitHub Actions hace el 11 y el 12.

#	Paso	Dónde	Detalle
1	Disparo	PC	Programador de tareas, todos los días a las 14:30 y al iniciar sesión si se perdió el horario
2	Ver qué fechas faltan	PC	Consulta el catálogo del portal y lo compara con lo ya procesado
3	Descargar	PC	Cada fecha que falte, con reintentos y verificando el tamaño contra el catálogo
4	Validar la fecha interna	PC	La carpeta `AAAA-MM-DD/` del zip tiene que coincidir con la fecha esperada
5	Limpiar y procesar	PC	DuckDB lee los 7 comercios, descarta el pie, controla claves repetidas y normaliza provincias
6	Escribir Parquet	PC	Cuatro archivos por día (precios, productos, sucursales y comercios) en `fecha=AAAA-MM-DD/`, unos 70 MB
7	Archivo histórico y registro	PC	Copia a la carpeta de Google Drive para escritorio y registra filas y sucursales por comercio
8	Agregar el día	PC	DuckDB calcula el agregado por fecha, comercio, producto y provincia, sin sucursales Web ni sin provincia: unas 1,4 M de filas. Columnas: mediana, mínimo y máximo del precio de lista, sucursales, y sucursales y mediana de cada promo

#	Paso	Dónde	Detalle
9	Carga en BigQuery	PC	Tabla raw particionada por fecha. Reemplaza la partición, así correr dos veces no duplica, y cada partición vence a los 14 días
10	Avisar a GitHub	PC	`repository_dispatch`, solo si cargó una fecha nueva
11	dbt snapshot y build	GitHub Actions	Corre dentro de BigQuery: historial de cambios, modelos y tests de calidad y de volumen
12	Exportar y publicar	GitHub Actions	Marts a JSON o Parquet, y publicación en GitHub Pages con la fecha del dato

Etapas y tareas

Las etapas 0 y 1 están cerradas. La 3 está construida y corre programada; se cierra después de una semana sin intervención. La 2 sigue con las decisiones que quedan.

Etapa 0: acceso

- [x] ~~Probar el acceso desde GitHub Actions y Google Colab (responden 403)~~
- [x] ~~Confirmar el acceso desde la PC (responde 200)~~
- [x] ~~Decidir desde dónde se descarga: la PC, con el procesamiento en la nube~~
- [ ] Enviar la consulta a soporte SEPA

Listo cuando: la PC baja el archivo tres días seguidos sin intervención. Se cumple al terminar la etapa 3.

Etapa 1: exploración y medición

- [x] ~~Estructura y formato del archivo, contra el Anexo II~~
- [x] ~~Volumen por día y por comercio, y elección de los 7 comercios~~
- [x] ~~Peso en CSV y en Parquet: 1.506 MB contra 70 MB por día~~
- [x] ~~Filas después de agregar, para cada grano~~
- [x] ~~Ritmo de cambios en una semana: 2,6 % de escrituras por día~~

Etapa 2: decisiones de diseño

- [x] ~~Las 5 preguntas, los 7 comercios, el precio de lista y el detalle completo en Drive~~

- [x] ~~Foto diaria o historial de cambios en BigQuery: historial, con snapshots de dbt~~
- [x] ~~Cuántos días puede faltar un producto sin que se cierre su precio: se ignoran las ausencias y a los 7 días pasa a "no disponible"~~
- [x] ~~Grano de `fct_precios_agregados`~~
- [x] ~~Cómo llegan los datos a BigQuery: la PC carga el agregado directo, sin Cloud Storage~~
- [x] ~~Horario: la PC a las 14:30 y el workflow cuando la PC avisa~~
- [ ] Criterio y tamaño de la canasta

Listo cuando: cada decisión tiene respuesta y justificación escrita.

Etapa 3: ingesta en la PC

- [x] ~~Instalar Google Drive para escritorio y crear la carpeta del proyecto~~
- [x] ~~Definir la estructura del repo (`ingesta/`) y las rutas como variables de entorno~~
- [x] ~~Ver qué fechas faltan: catálogo del portal contra lo ya procesado~~
- [x] ~~Descargar con reintentos y verificar el tamaño~~
- [x] ~~Validar la fecha interna del zip~~
- [x] ~~Procesar los 7 comercios a los tres Parquet, con los controles ya probados~~
- [x] ~~Normalizar provincias y contar las filas sin sucursal o sin provincia~~
- [x] ~~Guardar en Drive y borrar los temporales (zip y CSV)~~
- [x] ~~Registro diario con filas y sucursales por comercio, y alerta si un comercio varía más de ±5 %~~
- [x] ~~Script principal idempotente: correrlo dos veces no duplica nada~~
- [x] ~~Programarlo en el Programador de tareas~~

Listo cuando: hay una semana de archivos en Drive sin intervención manual.

Etapa 4: carga a BigQuery y dbt

- [ ] Protecciones de costos primero: facturación, alerta de presupuesto, cuota diaria y `maximum_bytes_billed`
- [ ] Cuenta de servicio para la PC con permiso solo para cargar en el dataset raw; la ruta de la clave, en una variable de entorno
- [ ] En la PC: calcular el agregado del día con DuckDB y cargarlo a BigQuery reemplazando la partición
- [ ] Aviso de la PC con `repository_dispatch` (token fine-grained solo para este repo, en una variable de entorno) y workflow con disparador manual
- [ ] GitHub Actions entra a Google Cloud con federación de identidad, sin clave

- [ ] dbt snapshot: historial de cambios con la última fecha vista y la tolerancia de 7 días; check_cols con todas las columnas menos sucursales
- [ ] Modelos dbt: dimensiones, hechos y marts
- [ ] Tests de calidad y de volumen por comercio

Listo cuando: `dbt build` termina en verde en el workflow que dispara la PC, y la factura del mes da $0 o centavos.

Etapa 5: el sitio

- [ ] Decidir cómo consulta los datos: JSON estáticos, Parquet con DuckDB-WASM o una API
- [ ] Exportar los marts
- [ ] Páginas para las 5 preguntas: canasta, provincias, comparación entre cadenas, promociones y evolución de cada producto
- [ ] Mostrar la fecha del dato

Listo cuando: el sitio público carga al instante y muestra la fecha del dato.

Etapa 6: documentación y portfolio

- [ ] README con la arquitectura y las decisiones
- [ ] Un documento por etapa
- [ ] Agregar el proyecto al portfolio

Listo cuando: alguien que no conoce el proyecto entiende qué hace y cómo, sin preguntar.

Decisiones

Hay once decisiones resueltas y cuatro pendientes. La próxima es el criterio de la canasta.

Decisión	Estado	Respuesta o etapa en que se cierra
Desde dónde se descarga el archivo	Resuelta	Desde la PC (opción B). El resto corre en BigQuery y GitHub Actions
Qué comercios se conservan	Resuelta	Los 7 más grandes, el 99 % de las filas
Qué preguntas responde el proyecto	Resuelta	Canasta, provincias, variación entre cadenas y sucursales, promociones y evolución de cada producto
Qué precio se usa	Resuelta	El de lista para las preguntas 1, 2, 3 y 5. Las promociones, solo para la 4

Decisión	Estado	Respuesta o etapa en que se cierra
Qué guarda el archivo en Drive	Resuelta	El detalle completo de las 7 cadenas en todo el país, unos 25 GB por año
Dónde corre el sitio	Resuelta	GitHub Pages como hosting
Foto diaria o historial de cambios	Resuelta	Historial de cambios con snapshots de dbt (SCD tipo 2), unos 1,2 GB por año. La canasta, como foto diaria
Tolerancia a ausencias de un producto	Resuelta	Se ignoran las ausencias: se guarda la última fecha vista y, a los 7 días sin aparecer, pasa a "no disponible"
Cómo llegan los datos a BigQuery	Resuelta	La PC calcula el agregado del día y lo carga directo, en una tabla raw con vencimiento de partición. Sin Cloud Storage: GitHub Actions no lee archivos, dbt corre dentro de BigQuery
Horario de la PC y del workflow	Resuelta	PC: 14:30 y al iniciar sesión. El workflow lo dispara la PC con `repository_dispatch` cuando carga una fecha nueva, más un disparador manual
Grano de `fct_precios_agregados`	Resuelta	Fecha × comercio × producto × provincia, sin Web ni sin provincia: ~1,37 M filas por día. El historial sigue los precios de lista y las promos, pero no la cantidad de sucursales, que varía a diario sin que cambie el precio: 3,25 % de cambios por día (7,81 % si la siguiera), unos 2 GB por año
Criterio y tamaño de la canasta	Pendiente	Etapa 2
Cómo se autentica cada parte en Google Cloud	Pendiente	Etapa 4. Propuesta: la PC con una cuenta de servicio de permisos mínimos; GitHub Actions con federación de identidad, sin clave
Cómo consulta datos el sitio	Pendiente	Etapa 5: JSON estáticos, Parquet con DuckDB-WASM o una API

Decisión	Estado	Respuesta o etapa en que se cierra
Qué pasa cuando vence el plan de Drive	Pendiente	Antes de que se cumpla un año

Riesgos y cómo se mitigan

El riesgo nuevo es que la PC pasa a ser parte del pipeline. Los 7 días que guarda el portal dan margen, pero hay que vigilarlo.

Riesgo	Qué pasaría	Mitigación
La PC queda apagada más de 7 días	Se pierden fechas para siempre	El script se pone al día solo con lo que el portal todavía guarda. Una alerta avisa si el último día procesado tiene más de 3 días
Se rompe el disco de la PC	Se pierde la historia acumulada	Los Parquet quedan también en Drive, el historial en BigQuery y el código en GitHub
Las credenciales quedan en la PC	Acceso a tus datos y a la facturación	Fuera del repo, en variables de entorno. La cuenta de servicio solo puede cargar en el dataset raw, y el token de GitHub solo sirve para este repo
Vence el token de GitHub (máximo un año)	La PC no puede avisar y el sitio deja de actualizarse	Anotar la fecha de vencimiento y renovarlo antes. Mientras tanto, el disparador manual sirve de respaldo
Falta un comercio o llega dañado	Un día con un hueco grande de datos	El proceso de ese día falla y avisa. Al día siguiente se reintenta, dentro de la ventana de 7 días
Un comercio informa de menos, como DIA el 13/10	Datos incompletos que pasan sin que nadie lo note	Registro diario con alerta si un comercio varía más de ±5 % contra el mismo día de la semana anterior, y test de volumen en dbt

Riesgo	Qué pasaría	Mitigación
El portal cambia el formato o las columnas	La lectura falla o lee mal	Validar las columnas esperadas antes de procesar, y fallar con un mensaje claro
GitHub desactiva los workflows programados tras 60 días sin actividad	BigQuery y el sitio dejan de actualizarse	El workflow no usa horario fijo: lo dispara la PC con `repository_dispatch`
Vence el plan de Drive (1 año)	No se pueden subir archivos nuevos	Planificar la mudanza del archivo histórico antes del vencimiento

Próximo paso

Definir el criterio y el tamaño de la canasta, y qué datos de producto se suben para mostrar nombres en el sitio. Mientras tanto, la etapa 3 corre sola: la primera ejecución programada es el `Oct 19, 2026` a las 14:30, y conviene revisar el log cada día durante esa semana.